Digitale Codierungen | DigCod

Zusammenfassung

INHALTSVERZEICHNIS

1. Stellenwertsystem ... 3
1.1. Konversion ... 3
1.2. Nachkommastellen ... 3
1.3. Subtraktion durch Addition ... 3

2. Dualsystem ... 4
2.1. Arithmetik ... 4
2.1.1. Komplementbildung (Zweierkomplement) ... 4
2.1.2. Subtraktion ... 4
2.1.3. Addition ... 4
2.1.4. Multiplikation ... 4
2.1.5. Division ... 4
2.2. Wertebereich ... 4
2.3. Bit ... 5
2.4. Präfixe ... 5
2.4.1. Multiplikation/Division ... 5

3. Codierungen ... 5
3.1. Warum reicht eine einzige Zahlendarstellung nicht aus? ... 5
3.2. Vorzeichen ... 6
3.2.1. Betrag mit Vorzeichen ... 6
3.2.2. (b-1) Komplement / Einerkomplement ... 7
3.2.3. (b) Komplement / Zweierkomplement ... 7
3.2.4. Exzess-Codierung (Bias-Code) ... 7
3.3. Gleitkommazahlen ... 8
3.3.1. Fixkommazahl ... 8
3.3.2. Allgemeiner Wertebereich ... 9
3.3.3. Auflösung ... 10
3.3.4. Arithmetik ... 10
3.4. Gleitkomma ... 10
3.4.1. Sonderfälle ... 12
3.4.2. Präzision ... 12
3.4.3. Addition ... 12
3.4.4. IEEE 754 - Single vs. Double ... 13
3.5. Text ... 13
3.5.1. ASCII ... 13
3.5.2. Unicode ... 14

4. Boolsche Logik ... 16
4.1. Terme ... 17
4.2. Normalformen ... 17
4.2.1. Disjunktive Normalform ... 17
4.3. NAND (Not AND) ... 17
4.4. Von Logik zur Arithmetik ... 17

4.5. Definition $\mathbb{Z}_2$ 18
4.5.1. Bitfolge Darstellungen 18
5. Qubit 19
6. CPU 19

1. STELLENWERTSYSTEM

Begriff	*Bedeutung*
Basis (Radix)	$E \in \mathbb{N}, R \geq 2$
Ziffernmenge	$T_R = \{0, 1, \ldots, R-1\}$
Darstellung einer Zahl (Polynom)	$N_R = \sum_{i=0}^{n} d_i R^i$
Stellenwertigkeit an der Position i	$d_i \in Z_R, R^i$

Beispiele

Dezimalsystem – $R = 10$ – $Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$ – 110_{10} = 0d110	Oktalsystem – $R = 8$ – $Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7\}$ – 110_8 = 0o110 = 72_{10}
Dualsystem – $R = 2$ – $Z_{10} = \{0, 1\}$ – 0110_2 = 0b0110 = 6_{10}	Hexadezimalsystem – $R = 16$ – $Z_{10} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F\}$ – $2D_{16}$ = 0x2D = 45_{10}

1.1. KONVERSION

Dezimal- zu Dualsystem:
- Rechts shift ist eine Division durch 2
- Es entsteht nur dann ein Rest, wenn die Bitstelle $2^0 = 1$ ist.

Beispiel: 25_{10}
Resultat: $0001\ 1001_2$

Zähler	*Nenner*	*Resultat*	*Rest*	
25	2	12	**1**	↑
12	2	6	**0**	
6	2	3	**0**	
3	2	1	**1**	
1	2	0	**1**	

Dual- zu Dezimalsystem bei Nachkommastellen:
- Links shift ist eine Multiplikation mit 2

Beispiel: $0.187'5_{10}$
Resultat: $0.001\ 1_2$

a	*b*	*Resultat*	*Rest*	
0.1875	2	0.375	**0**	↓
0.375	2	0.75	**0**	
0.75	2	1.5	**1**	
0.5	2	1	**1**	

1.2. NACHKOMMASTELLEN

Erweiterung der Darstellung einer Zahl: $N_R = d_n R^n + \ldots + d_{10} R^0 + d_{-1} R^{-1} + \ldots + d_{-m} R^{-m}$

Beispiel: $(101.01)_2 = 1 * 2^2 + 0 * 2^1 + 1 * 2^0 + 0 * 2^{-1} + 1 * 2^{-2} = 4 + 1 + \frac{1}{4} = 5.25_{10}$

1.3. SUBTRAKTION DURCH ADDITION

Beispiel: $753 + 247 = 1000$

Bei 1000 einen Überlauf (mod 1000): $753 + 247 \equiv 0 \Leftrightarrow 753 \equiv -247 \bmod 1000$

Dann wäre $620 - 247 \equiv 620 + 753 = 1373 \equiv 373 \bmod 1000$

2. DUALSYSTEM

2.1. ARITHMETIK

2.1.1. Komplementbildung (Zweierkomplement)

1) Rechnen in n Bit $\Rightarrow \bmod 2^n$
   - Übertrag aus dem MSB wird verworfen (gerechnet wird in $\mathbb{Z}_{2^n}$)
2) $-b$ als Zweierkomplement
   - Additives Inverses von $b \bmod 2^n$ ist: $-b \equiv 2^n - b \bmod 2^n$
   - Praktische Berechnung:
     - Bits invertieren: $\sim b$
     - $+1$ addieren
     - $2K(b) = \sim b + 1$

2.1.2. Subtraktion

Subtraktion wird durch Addition des Komplements ersetzt

$$a - b \equiv a + 2K(b) \bmod 2^n$$

Beispiel 1 : $13_{10} - 5_{10}$

8 Bit, wobei $13_{10} = 0000\ 1101_2, 5_{10} = 0000\ 0101_2$
Zweierkomplement von 5_{10}:
- invertieren: $1111\ 1010_2$
- $+1 : 1111\ 1011_2$

Addieren: $0000\ 1101_2 + 1111\ 1011_2 = 0001\ 0000\ 1000_2$

MSB-Übertrag weg: $0000\ 1000_2 = 8_{10}$

Also: $13_{10} - 5_{10} = 8_{10}$

2.1.3. Addition

TODO:
signed, unsigned

2.1.4. Multiplikation

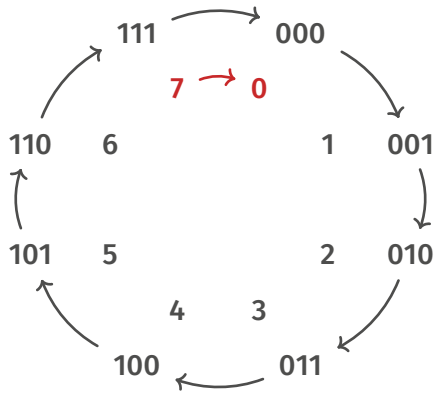
TODO:
signed, unsigned

2.1.5. Division

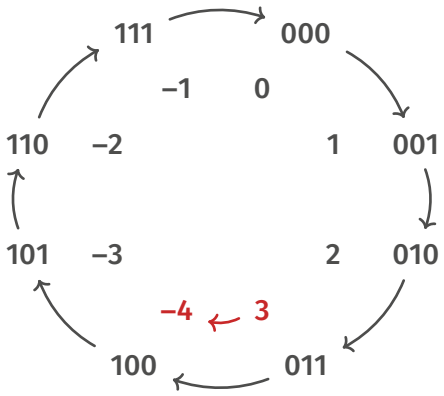
TODO:
signed, unsigned

2.2. WERTEBEREICH

Graphische Veranschaulichung für Wortbreite von 3 Bit



unsigned: Überlauf von **7** auf **0**



signed: Überlauf von **3** auf **–4**

2.3. BIT

Begriff	*Bedeutung*
set bit (gesetztes Bit)	**1**
cleared bit (gelöschtes Bit)	**0**
LSB	Least Significant Bit (Bit **0**)
MSB	Most Significant Bit (Bit $n - 1$) in einer n-stelligen Binärzahl
Nibble	Binärzahl mit 4 Bit
Oktett	Binärzahl mit 8 Bit
Byte	Oktett

2.4. PRÄFIXE

Binär	*Näherung*	*Binärpräfix*	*Dezimal*	*Dezimalpräfix*
2^{10}	$1.024 \cdot 10^3$	Ki - Kibi	10^3	K - Kilo
2^{20}	$1.049 \cdot 10^6$	Mi - Mebi	10^6	M - Mega
2^{30}	$1.074 \cdot 10^9$	Gi - Gibi	10^9	G - Giga
2^{40}	$1.100 \cdot 10^{12}$	Ti - Tebi	10^{12}	T - Tera
2^{50}	$1.126 \cdot 10^{15}$	Pi - Pebi	10^{15}	P - Peta
2^{60}	$1.153 \cdot 10^{18}$	Ei - Exbi	10^{18}	E - Exa

2.4.1. Multiplikation/Division

1) Umformen in Potenzschreibweise
2) Addition der Exponenten
3) Umformen in Präfixschreibweise

Beispiel: $128K \cdot 64M = 2^7 \cdot 2^{10} \cdot 2^6 \cdot 2^{20} = 2^{17+26} = 2^{43} = 2^3 \cdot 2^{40} = 8T$

3. CODIERUNGEN

Erst wenn man die Codierung kennt, kann man daten richtig interpretieren.

3.1. WARUM REICHT EINE EINZIGE ZAHLENDARSTELLUNG NICHT AUS?

- Negative Zahlen
  - Vorzeichen-Bit
  - Einerkomplement
  - Exzess-Code

- Sehr grosse und kleine Zahlen
  - Fixkomma ist unflexibel
  - 334 Milliarden + 0.00001
- Genauigkeit
  - 0.1 ist im Dualsystem nicht exakt darstellbar
  - Rundungsfehler unvermeidbar
- Nicht nur Zahlen
  - Textdarstellung
  - Zeichenkodierungen

3.2. VORZEICHEN

Jede Codierung optimiert eine andere Eigenschaft

- Betrag mit Vorzeichen: Intuition
- Einerkomplement: Einfache Negation
- Zweierkomplement: Arithmetische Effizienz
- Exzess: Vergleichbarkeit/Sortierung

Binär	*Betrag mit V.*	*EinerK.*	*ZweierK.*	$C_{ex,-8,4}$
0000	0	0	0	-8
0001	1	1	1	-7
0010	2	2	2	-6
0011	3	3	3	-5
0100	4	4	4	-4
0101	5	5	5	-3
0110	6	6	6	-2
0111	7	7	7	-1
1000	0	-7	-8	0
1001	-1	-6	-7	1
1010	-2	-5	-6	2
1011	-3	-4	-5	3
1100	-4	-3	-4	4
1101	-5	-2	-3	5
1110	-6	-1	-2	6
1111	-7	0	-1	7

3.2.1. Betrag mit Vorzeichen

Erstes Bit signalisiert, ob Zahl positiv (**0**) oder Negativ (**1**) ist.

Problem: Bekannte Rechenregeln funktionieren nicht.

$$-19_{10} \quad + 1_{10} \quad = -18_{10}$$
$$1001\ 0011_2 + 0000\ 0001_2 = 1001\ 0100_2 = -20_{10}$$

Beispiel 2 : **Codierung**

Gegeben	*Dezimal → Binär*	*Ergebnis*
$x = -3_{10}$	$3_{10} = x011_2$	$-3_{10} = 1011_2$

Beispiel 3 : **Decodierung**

Gegeben	*Binär → Dezimal*	*Ergebnis*
$x = 1001_2$	$x001_2 = 1_{10}$	$1001_2 = -1_{10}$

3.2.2. (b-1) Komplement / Einerkomplement

Von allen bits wird das Komplement gebildet.

Problem: $0000\ 00000_2 = 1111\ 1111_2 = 0_{10}$

Beispiel 4 : **Codierung**

Gegeben	*Dezimal → Binär*	*Komplement*	*Ergebnis*
$x = -5_{10}$	$5_{10} = 0101_2$	$0101_2 \Rightarrow 1010_2$	$-5_{10} = 1010_2$

Beispiel 5 : **Decodierung**

Gegeben	*Komplement*	*Binär → Dezimal*	*Ergebnis*
$x = 1011_2$	$1011_2 \Rightarrow 0100_2$	$0100_2 = 4_{10}$	$1011_2 = -4_{10}$

3.2.3. (b) Komplement / Zweierkomplement

Nach der Komplementbildung wird **1** addiert.

Beispiel 6 : **Codierung**

Gegeben	*Dezimal → Binär*	*Komplement*	*+1*	*Ergebnis*
$x = -5_{10}$	$5_{10} = 0101_2$	$0101_2 \Rightarrow 1010_2$	$0101_2 \Rightarrow 1011_2$	$-5_{10} = 1011_2$

Beispiel 7 : **Decodierung**

Gegeben	*−1*	*Komplement*	*Binär → Dezimal*	*Ergebnis*
$x = 1111_2$	$1111_2 \Rightarrow 1110_2$	$1110_2 \Rightarrow 0001_2$	$0001_2 = 1_{10}$	$1111_2 = -1_{10}$

3.2.4. Exzess-Codierung (Bias-Code)

Darstellung vorzeichenbehafteter Zahlen durch **Verschiebung des Wertebereichs**.

Statt negativer Zahlen wird ein **Offset (Bias)** addiert

Begriff 1 : $C_{ex,-B,n}(x)$

C_{ex} = Exzess-Codierung
$-B$ = Bias (negativer Überhang zur 0)
n = Länge der binären Schreibweise
x = zu codierende Zahl

Gespeichert wird: $c = x + B$

Decodierung: $x = c - B$

Gegeben	Codierung	Decodierung
Basis $b = 2$ Wortbreite n Bias B typischerweise: $B = 2^{n-1} - 1$	$C_{ex,-B,n}(x) = x + B$ mit $x \in [-B, 2^n - 1 - B]$	$x = c - B$

Beispiel 8 : **Codierung**

Gegeben

Wortbreite: $n = 8$ Bit
Bias: $B = 2^7 - 1 = 127$
Zu codierende Zahl: $x = -5$

Bias addieren

$c = x + B = -5 + 127 = 122$

Dezimal → Binär

$122_{10} = 0111\ 1010_2$

Ergebnis

$C_{ex,-127,8}(-5) = 0111\ 1010_2$

Beispiel 9 : **Decodierung**

Gegeben

Wortbreite: $n = 8$ Bit
Bias: $B = 2^7 - 1 = 127$
Bitmuster: $C_{ex,-127,8} = 0111\ 1010_2$

Binär → Dezimal

$0111\ 1010_2 = 122_{10}$

Bias subtrahieren

$x = c - B = 122 - 127 = -5$

Ergebnis

$0111\ 1010_2 \Rightarrow -5_{10}$

3.3. GLEITKOMMAZAHLEN

3.3.1. Fixkommazahl

Skalierte Ganzzahl

Pro	Contra
Einfache Implementierung	Fester Wertebereich

Deterministische Genauigkeit	Feste Auflösung
Keine Exponentendarstellung	Überlauf bei grossen Zahlen
Schnelle Berechnung	Ungeeignet für stark unterschiedliche Grössenordnungen

Begriff 2 : $C_{FK,k,n}(x) = x \cdot 2^k$

C_{FK} = Fixkomma-Codierung
k = Anzahl Nachkommabits
n = Länge der binären Schreibweise - daraus ergibt sich die Anzahl der Vor-Kommastellen: $n - k$
x = zu codierende Zahl
I = Ganzzahl

Beispiel 10 : Codierung

Gegeben

Wortbreite: $n = 8$ Bit
Nachkommabits: $k = 4$
Zu codierende Zahl: $x = 3.25$

Skalieren

$I = x \cdot 2^k = 3.25 \cdot 16 = 52$

Dezimal → Binär

$52_{10} = 0011\ 0100_2$

Ergebnis

$C_{FK,4,8}(3.25) = 0011\ 0100_2$

Beispiel 11 : Decodierung

Gegeben

Wortbreite: $n = 8$ Bit
Nachkommabits: $k = 4$
Bitmuster: $C_{FK,4,8}(3.25)$

Binär → Dezimal

$0011\ 0100_2 = 52_{10} = I$

Skalieren

$x = \frac{I}{2^k} = \frac{52}{16} = 3.25$

Ergebnis

$0011\ 0100_2 \Rightarrow 3.25_{10}$

3.3.2. Allgemeiner Wertebereich

Wertebereich bei n Bit und k Nachkommabits

	Ganzzahlbereich	Reeller Bereich
unsigned	$I \in [0, 2^n - 1]$	$x \in \left[0, \frac{2^n - 1}{2^k}\right]$
signed	$I \in \left[-2^{n-1}, 2^{n-1} - 1\right]$	$x \in \left[-\frac{2^{n-1}}{2^k}, \frac{2^{n-1} - 1}{2^k}\right]$

***Beispiel** 12*

Gegeben

Wortbreite: $n = 8$ Bit
Nachkommabits: $k = 3$
Darstellung im Zweierkomplement (= inkl. Vorzeichen)

Ganzzahlbereich

$I \in \left[-2^{8-1}, 2^{8-1} - 1\right] = [-128, 127]$

Reeller Bereich

$$x = \frac{I}{2^k} = \frac{I}{8}$$

$$x_{\text{min}} = \frac{-128}{8} = -17$$

$$x_{\text{max}} = \frac{127}{8} = 15.875$$

Ergebnis

$x \in [-16, 15.875]$

3.3.3. Auflösung

Kleinster darstellbarer Schritt: $\Delta x = 2^{-k}$

Absoluter Fehler: $E_{\text{abs}} = \left|x_{\text{korrekt}} - x_{\text{gerundet}}\right|$

Relativer Fehler: $E_{\text{rel}} = \frac{\left|x_{\text{korrekt}} - x_{\text{gerundet}}\right|}{x_{\text{korrekt}}}$

3.3.4. Arithmetik

Addition, Subtraktion und Zweierkomplement sind wie bei Ganzzahlen

– Addiert man zwei Fixkommazahlen $z_0 + z_1$ mit $k_0 > k_1$, so muss z_1 um $k_0 - k_1$ Bits nach rechts geschoben werden.

Multiplikation und Division verschieben das Komma

– Multipliziert man zwei Fixkommazahlen $z_0 \cdot z_1 = z_2$, dann ist $k_2 = k_0 + k_1$

k muss für jede Zahl berücksichtigt werden

– Wird von den meisten Programmiersprachen kaum unterstützt
– Meist von Hand in Kommentaren
– Limitiert die Zahlen auf Bereiche, die zur Compilezeit bekannt sind
– unflexibel und fehleranfällig, aber performant

3.4. GLEITKOMMA

***Begriff** 3 :* $\pm m \cdot 2^e$

m = Mantisse (Signifikand)
e = Exponent

← 32 Bit →		
±	8 – Bit Exponent	23 – Bit Mantisse

$$\pm(1 + \text{Mantisse}) \cdot 2^{\text{Exponent} - 127}$$

Beispiel 13 : Codierung

Gegeben

Zu codierende Zahl: $x = -42.625$

Vorzeichenbit bestimmen

x ist negativ, somit eine **1**

Mantisse Dezimal → Binär

$$\underbrace{101010}_{42}.\underbrace{101}_{0.625}$$

Komma verschieben

$101010.101_2 \cdot 2^0 = 1.01010101_2 \cdot 2^5$

Runden

Bei Überfluss: falls vorherige Bit eine **1** ist, aufrunden, ansonsten kürzen (in diesem Beispiel nicht nötig).

Bias addieren

$5 + 127 = 132$

Exponent Dezimal → Binär

$132_{10} \Rightarrow 1000\ 0100_2$

Zusammensetzen

1 | 1000 0100 | (1)010 1010 1000 0000 0000 0000
⇒ 1100 0010 0010 1010 1000 0000 0000 0000_2

Beispiel 14 : Decodierung

Gegeben

Bitmuster: 0100 0001 0011 0110 0000 0000 0000 0000_2

Komponenten extrahieren

0 | 1000 0010 | (1)011 0110 0000 0000 0000 0000

Erstes Bit

0 = Zahl ist positiv

Exponent Binär → Dezimal

$1000\ 0010_2 \Rightarrow 130_{10}$

Bias subtrahieren

$130 - 127 = 3$

Mantisse Binär → Dezimal

$011\ 0110\ 0000\ 0000\ 0000\ 0000_2 \Rightarrow 0.40625_{10}$

Zusammensetzen

$+1 \cdot (1 + 0.40625) \cdot 2^3 = 11.25$

3.4.1. Sonderfälle

Fall	*Vorzeichenbit*	*Exponent*	*Mantisse*	*Beispielwert*
Positive Null	**0**	**0000 0000**	**000 0000 0000 0000 0000 0000**	**+0**
Negative Null	**1**	**0000 0000**	**000 0000 0000 0000 0000 0000**	**−0**
Positive Unendlichkeit	**0**	**1111 1111**	**000 0000 0000 0000 0000 0000**	**+∞**
Negative Unendlichkeit	**1**	**1111 1111**	**000 0000 0000 0000 0000 0000**	**−∞**
NaN	**0** oder **1**	**1111 1111**	mindestens ein Bit **1**	Nicht darstellbare Werte
Subnormale Zahlen	**0** oder **1**	**0000 0000**	mindestens ein Bit **1**	$\approx 1.4 \cdot 10^{-45}$ (positiv)
Normalisierte Zahlen	**0** oder **1**	alles ausser **0000 0000** und **1111 1111**	beliebig	Alle regulären Werte

3.4.2. Präzision

Viele Zahlen können nicht präzise dargestellt werden

- $0.1_{10} = 0.0001100110011...$
  - Abbruch nach 23 Bits
  - Beispiel: $0.1 + 0.2 \neq 0.3$
- Präzision für «Single Precision»: $\varepsilon \approx 2^{-23} \approx 10^{-7}$
  - ε ist die kleinste relative Differenz nahe 1

3.4.3. Addition

1) Hidden Bit ergänzen
2) Exponenten vergleichen
   - Wenn unterschiedlich: Bei kleinerer Zahl Mantisse nach rechts schieben
3) Vorzeichen berücksichtigen
   - Gleiche Vorzeichen: Addieren
   - Unterschiedliche Vorzeichen: Subtrahieren
4) Addition/Subtraktion der Signifikanden
5) Falls Carry = 1: Normalisieren
   - Erhöhe Exponent um 1
   - Schiebe Signifikand um 1 nach rechts
6) Hidden Bit entfernen

***Beispiel** 15 :* $x = 1.5, y = 0.75$

1) Hidden Bit ergänzen

$$x' = 0 \mid 0111\ 1111 \mid (1)100\ 0000\ 0000\ 0000\ 0000\ 0000$$
$$y' = 0 \mid 0111\ 1110 \mid (1)100\ 0000\ 0000\ 0000\ 0000\ 0000$$

1) Exponent verschieben bei y'

$$x'' = 0 \mid 0111\ 1111 \mid (1)100\ 0000\ 0000\ 0000\ 0000\ 0000$$
$$y'' = 0 \mid 0111\ 1111 \mid (0)110\ 0000\ 0000\ 0000\ 0000\ 0000$$

1) Vorzeichen: beide positiv → Addition
2) Addition $z'' = x'' + y''$

$$z'' = 0 \mid 0111\ 1111 \mid (10)010\ 0000\ 0000\ 0000\ 0000\ 0000$$

1) Carry = 1: Normalisieren

$$z' = 0 \mid 1000\ 0000 \mid (1)001\ 0000\ 0000\ 0000\ 0000\ 0000$$

1) Hidden Bit entfernen

$$z = 0 \mid 1000\ 0000 \mid 001\ 0000\ 0000\ 0000\ 0000\ 0000$$

3.4.4. IEEE 754 - Single vs. Double

Format	*Bits*	*Vorzeichen*	*Exponent*	*Mantisse*	*Bias*
Single	32	1	8	23	−127
Double	64	1	11	52	−1023

- Grösserer Exponent → grösserer Wertebereich
- Grössere Mantisse → höhere Präzision
  - Single Precision: $\varepsilon \approx 2^{-23} \approx 10^{-7}$
  - Double Precision: $\varepsilon \approx 2^{-52} \approx 10^{-16}$

• Interpretation
- Single ≈ 7 signifikante Dezimalstellen
- Double $\approx 15^{-16}$ signifikante Dezimalstellen

3.5. TEXT

- Text besteht aus einer endlichen Folge von Zeichen: Buchstaben, Zahlen, Satzzeichen usw.
- Alle Zeichen stammen aus einer endlichen Menge Z, dem Zeichensatz (character set)
  - Jedem Zeichen kann eindeutig eine natürliche Zahl zugeordnet werden
- Ein Encoding (character encoding, Zeichenkodierung) ist eine bijektive Funktion E, die jedem Zeichen z eine natürliche Zahl zuordnet
  - $E : Z \to \{0, 1, \ldots, |Z| - 1\}$
  - Text mit n Zeichen $z_0, \ldots, z_{n-1}$ kann als endliche Folge von kodierten Zeichen beschrieben werden

3.5.1. ASCII

American Standard Code for Information Interchange

- Grösse des Zeichensatzes: $2^7 = 128 \to 7$ **Bit** (nicht 8 Bit!)
- 8-Bit-ASCII sind Extended ASCII-Varianten und nicht standardisiert, sondern Codepages – bspw.: – ISO-8859-1 (Latin-1)
  - für westeuropäische Sprachen mit zusätzlichen Buchstaben wie ä, ö, ü, é usw.

  – Windows-1252
  - Microsoft-Codepage – weitgehend wie ISO-8859-1 mit typografischen Zeichen wie €, “ usw.

  - Codepage 437
  - IBM-PC-Zeichensatz mit grafischen Symbolen, Linienzeichen und Sonderzeichen
- Enthält druckbare (darstellbare) Zeichen und (nicht darstellbare) Steuerzeichen (0x00=NUL, 0x07=BEL, …)

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	**NUL** 00	SOH 01	STX 02	ETX 03	EOT 04	ENQ 05	ACK 06	BEL 07	**BS** 08	**TAB** 09	**LF** 0A	VT 0B	FF 0C	**CR** 0D	SO 0E	SI 0F
1	DLE 10	DC1 11	DC2 12	DC3 13	DC4 14	NAK 15	SYN 16	ETB 17	CAN 18	EM 19	SB 1A	ESC 1B	FS 1C	GS 1D	RS 1E	US 1F
2	**SP** 20	**!** 21	**“** 22	**#** 23	**$** 24	**%** 25	**&** 26	**‘** 27	**(** 28	**)** 29	***** 2A	**+** 2B	**,** 2C	**–** 2D	**.** 2E	**/** 2F
3	**0** 30	**1** 31	**2** 32	**3** 33	**4** 34	**5** 35	**6** 36	**7** 37	**8** 38	**9** 39	**:** 3A	**;** 3B	**<** 3C	**=** 3D	**>** 3E	**?** 3F
4	**@** 40	**A** 41	**B** 42	**C** 43	**D** 44	**E** 45	**F** 46	**G** 47	**H** 48	**I** 49	**J** 4A	**K** 4B	**L** 4C	**M** 4D	**N** 4E	**O** 4F
5	**P** 50	**Q** 51	**R** 52	**S** 53	**T** 54	**U** 55	**V** 56	**W** 57	**X** 58	**Y** 59	**Z** 5A	**[** 5B	**** 5C	**]** 5D	**^** 5E	**_** 5F
6	**`** 60	**a** 61	**b** 62	**c** 63	**d** 64	**e** 65	**f** 66	**g** 67	**h** 68	**i** 69	**j** 6A	**k** 6B	**l** 6C	**m** 6D	**n** 6E	**o** 6F
7	**p** 70	**q** 71	**r** 72	**s** 73	**t** 74	**u** 75	**v** 76	**w** 77	**x** 78	**y** 79	**z** 7A	**{** 7B	**|** 7C	**}** 7D	**~** 7E	**DEL** 7F

3.5.2. Unicode

Globale Zeichennummerierung

- ASCII-kompatibel
- 1 bis 4 Byte pro Zeichen
- Häufige (westliche) Zeichen kurz
- Seltene (westliche) Zeichen länger
- Das erste Byte verrät, wie viele Bytes folgen
- Unicode = Codepoint
- UTF-8 = Bytecodierung

3.5.2.1. Struktur

Codepoint-Bereich	*Bytes*	*Bitmuster*
U+0000 - U+007F	1	0xxxxxxx
U+0080 - U+07FF	2	110xxxxx 10xxxxxx
U+0800 - U+FFFF	3	1110xxxx 10xxxxxx 10xxxxxx
U+10000 - U+10FFFF	4	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

- Erstes Byte zeigt Länge
- Folgebytes beginnen mit **10**
- ASCII-Zeichen (0-127) bleiben unverändert

3.5.2.2. Codierung

Gegeben: Unicode-Codepoint U

1) Bestimme den Wertebereich → Anzahl Bytes.
2) Schreibe U in Binärdarstellung.
3) Fülle die Bits in die x-Positionen des passenden Musters ein.
4) Wandle in Hex um.

***Beispiel** 16*

Gegeben

Zeichen: ä

Unicode

***U* = $E4_{16}$ = 228_{10} = $1110\ 0100_2$**

Benötigt 8 Bit → 2 Bytes

Muster einfügen

```
110xxxxx 10xxxxxx ⇒
11000011 10100100 ⇒ 1100 0011 1010 0100₂ = C3 A4₁₆
```

UTF-8(ä) = **$C3\ A4_{16}$**

3.5.2.3. Decodierung

Gegeben: Bytefolge

1) Lies das erste Byte.
2) Bestimme anhand der führenden Bits die Länge.
3) Entferne die Strukturpräfixe (110, 10, etc.).
4) Füge die restlichen Bits zusammen.
5) Interpretiere als Codepoint.

***Beispiel** 17*

Gegeben

$1110\ 0010\ 1000\ 0010\ 1010\ 1100_2$

Muster entfernen

```
11100010 10000010 10101100
1110xxxx 10xxxxxx 10xxxxxx ⇒
    0010   000010   101100 ⇒ 0010 0000 1010 1100₂ = 20 AC₁₆ = €
```

3.5.2.4. Encodings

UTF-16

– Meist 2 Bytes pro Zeichen
– Zeichen ausserhalb der BMP (Basic Multilingual Pane) benötigen in UTF-16 sogenannte Surrogate Pairs (4 Bytes)
– Nicht ASCII-kompatibel

Vorteil:

– Häufig effizient für asiatische Sprachen

Nachteil:

– Komplexere Handhabung

UTF-32

– Immer 4 Bytes pro Zeichen
– Direkter Zugriff auf Codepoints
– Sehr speicherintensiv

Encoding	*Länge*	*Vorteil*	*Nachteil*
UTF-8	1-4 Byte	ASCII-Kompatibel, effizient	Variable Länge
UTF-16	2-4 Byte	Teils effizient	Surrogate-Komplexität
UTF-32	4 Byte	Sehr einfach	Hoher Speicherbedarf

3.5.2.5. Basic Multilingual Plane (BMP)

Unicode ist als Zahlenraum definiert: $\mathbf{0 \leq U \leq 10\ FF\ FF_{16}}$

Das sind $\mathbf{1'114'111_{10}}$ mögliche Codepoints. Dieser Raum ist in Planes (Ebenen) aufgeteilt.

Die BMP umfasst $\boldsymbol{U}$ **+ 0000** bis $\boldsymbol{U}$ **+ FFFF**, also: $\mathbf{0 \leq U \leq 65535}$. Das sind genau 16 Bit.

In der BMP liegen:

– ASCII
– Lateinische Schriftzeichen
– Griechisch
– Kyrillisch
– Hebräisch
– Arabisch
– Viele asiatische Zeichen
– Mathematische Symbole
– Satzzeichen

4. BOOLSCHE LOGIK

– Eine Aussage ist entweder wahr (**1**) oder falsch (**0**)
– Aussagen können logisch verknüpft werden

Kommutativität

– $x \wedge y = y \wedge x$
– $x \vee y = y \vee x$

Assoziativität

– $(x \wedge y) \wedge z = x \wedge (y \wedge z)$
– $(x \vee y) \vee z = x \vee (y \vee z)$

Distributivität

– $x \wedge (y \vee z)$
$= (x \wedge y) \vee (x \wedge z)$

Absorption

– $x \vee (x \wedge y) = x$
– $x \wedge (x \vee y) = x$

($x \wedge y$ schreibt man auch als xy oder $x \cdot y$ oder $x \cap y$, $x \vee y$ als $x + y$ oder $x \cup y$)

x	y	$x \wedge y$	$x \vee y$	$x \oplus y$	$\neg x$
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Dualitätsprinzip: Ersetze in einer wahren Gleichung $\wedge \leftrightarrow \vee$ und $\mathbf{0} \leftrightarrow \mathbf{1}$, Ergebnis bleibt wahr.

TODO:
[De-Morgan-Gesetze]

4.1. TERME

Begriff	*Bedeutung*
Literal	Variable oder Negation einer Variablen: x_3 (positiver Literal), $\neg x_3$ oder $\overline{x_3}$ (negatives Literal)
Konjunktionsterm	Konjunktion von Literalen: $\overline{x_1}x_3x_4 = \overline{x_1} \wedge x_3 \wedge x_4$
Disjunktionsterm	Disjunktion von Literalen: $\overline{x_1} \vee x_3 \vee x_4$
Minterm	Konjunktionsterm, der alle Parameter der Funktion enthält
Maxterm	Disjunktionsterm, der alle Parameter der Funktion enthält

4.2. NORMALFORMEN

Standardisierte Schreibweise. Vorteile:

- aus Wahrheitstabelle konstruierbar
- Vereinfachung systematisch möglich
- Umsetzung als Schaltung (AND-OR-Structure) direkt ableitbar

Wir brauchen: DNF / KDNF als «Startpunkt» für KV-Diagramm:

4.2.1. Disjunktive Normalform

in Term ist in DNF, wenn er eine ODER-Verknüpfung von UND-Verknüpfungen ist, z.B.:

$$f(x,y,z) = (\neg x \wedge y) \vee (x \wedge z) = \neg x \wedge y \vee x \wedge z$$

Da $\wedge$ eine höhere Bindungsstärke als $\vee$ hat, sind die Klammern nicht zwingend notwendig. Die kanonische DNF (KDNF) ist die Disjunktion der Minterme.

4.3. NAND (NOT AND)

Universalbaustein

$$x \underbrace{|}_{\text{Sheffer stroke}} y = \overline{x \wedge y} = \overline{xy}$$

- praktisch (in CMOS schnell sowie einfach aufzubauen)
- funktional vollständig: jede Boolesche Funktion lässt sich nur mit NAND realisieren.

x	y	$x \wedge y$	$x \mid y$
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

OP	*impl*
NOT	$\overline{x} = \overline{x \wedge x} = x \mid x$
AND	$x \wedge y = \overline{x \mid y} = (x \mid y) \mid (x \mid y)$
OR	$x \vee y = \overline{\overline{x \vee y}} = \overline{\overline{x} \wedge \overline{y}} = \overline{(x \mid x) \wedge (y \mid y)} = (x \mid x) \mid (y \mid y)$

4.4. VON LOGIK ZUR ARITHMETIK

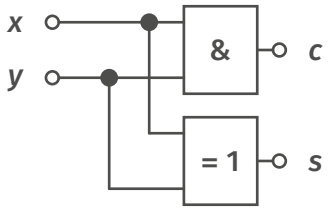
Boolesche Logik realisiert binäre Addition

XOR bildet die Addition zweier Bits ab (im Zahlenraum mit nur **0** und **1**, also **mod 2**), AND bildet den Übertrag ab

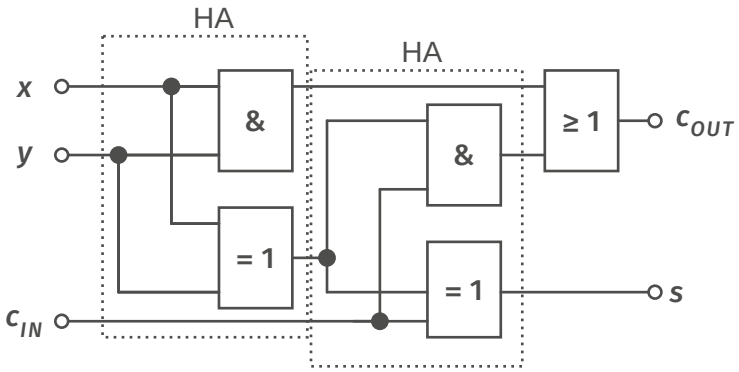
- $s = x \oplus y \rightarrow$ Addition mod 2
- $c = x \wedge y \rightarrow$ Übertrag

x	y	$x + y$	$x \wedge y$	$x \oplus y$
0	0	00	0	1
0	1	01	0	1
1	0	01	0	1
1	1	10	1	0

Halbaddierer (half adder)
- Kann 2 einstellige Binärzahlen addieren
- Hat 2 Eingänge (x, y) und 2 Ausgänge (s, c)



Volladdierer (full adder)
- Hat 3 Eingänge (x, y, c_{IN})



4.5. DEFINITION $\mathbb{Z}_2$

$$\mathbb{Z}_2 = (\{0, 1\}, +, \cdot)$$

- Abgeschlossen: Jede Operation erzeugt wieder ein Element in $\mathbb{Z}_2$
- Es existiert das neutrale Element (**0** für die Addition, **1** für die Multiplikation)
- Jedes Element ist sein eigenes additives Inverses.
- Addition und Multiplikation sind kommutativ und assoziativ.
- Es gilt das Distributivgesetz.

4.5.1. Bitfolge Darstellungen

Darstellung	*Beispiel*
Zahl	$0101_2 = 5_{10}$
Vektor	$(1, 0, 1)^T$
Polynom	$u^2 + 1$

***Beispiel** 18* : **Bitfolge als Vektor in $\mathbb{Z}_2^n$**

$$v, w \in \mathbb{Z}_2^4, v = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix}, w = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 1 \end{pmatrix}$$

$$v + w = \begin{pmatrix} 1 \\ 0 \\ 1 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \\ 0 \\ 1 \end{pmatrix} \equiv \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \end{pmatrix} \bmod 2$$

4.5.1.1. Polynom

Polynome erlauben

- Verschiebungen elegant zu beschreiben
- Redundanz strukturiert zu erzeugen
- Generatorpolynome (zyklische Codes)
- CRC-Rechnung als Polynomdivision

Abbildung:

5. QUBIT

- Superposition
  - Ein Qubit kann gleichzeitig **0** und **1** repräsentieren. Mehrere Qubits können dadurch alle 2^n möglichen Zustände gleichzeitig darstellen.
  - $|\Psi\rangle = \alpha\ |0\rangle + \beta\ |1\rangle$
  - $P(0) = |\alpha|^2, P(1) = |\beta|^2$
- Beispiel
  - $|\Psi\rangle = \sqrt{0.75}\ |0\rangle + \sqrt{0.25}\ |1\rangle$
  - $P(0) = 0.75, P(1) = 0.25$
- Interferenz
  - α, β sind Amplituden. Zwei Qubits können sich gegenseitig beeinflussen, indem die Amplituden mittels Interferenz verstärkt oder ausgelöscht werden
  - Quantenalgorithmen verändern gezielt die Amplituden:
    - richtige Lösungen werden verstärkt
    - falsche Lösungen werden abgeschwächt
- Verschränkung
  - Verschränkung bedeutet, dass zwei Qubits einen gemeinsamen Zustand besitzen, der sich nicht in zwei unabhängige Einzelzustände zerlegen lässt.
  - Misst man eines der Qubits, ist der Zustand des anderen sofort festgelegt

6. CPU

Gesamtübersicht des CPU Zyklus

1) Fetch: Instruktion aus RAM lesen
2) Decode: Opcode interpretieren
3) Operand-Fetch: Speicherwerte laden
4) Execute: ALU rechnet
5) Writeback: Ergebnis ins Register
6) IP erhöhen